

HLD $\Rightarrow$ High level Design $\Rightarrow$ Architecture

↓

impl for HLD is good but very diff and not mandatory

↓

discuss the design

→ understanding fundamentals vvi^{imp}

→ discussion is vvi^{imp}

{ Sample Question: Coinbase dk code od in Superbowl. }

$\Rightarrow$ Why system design, why distributed system?

ex $\Rightarrow$ DELICIOUS $\Rightarrow$ del.icio.us

ff $\rightarrow$ 2004 $\Rightarrow$ had no cloud support, so syncing of data across devices was not possible.

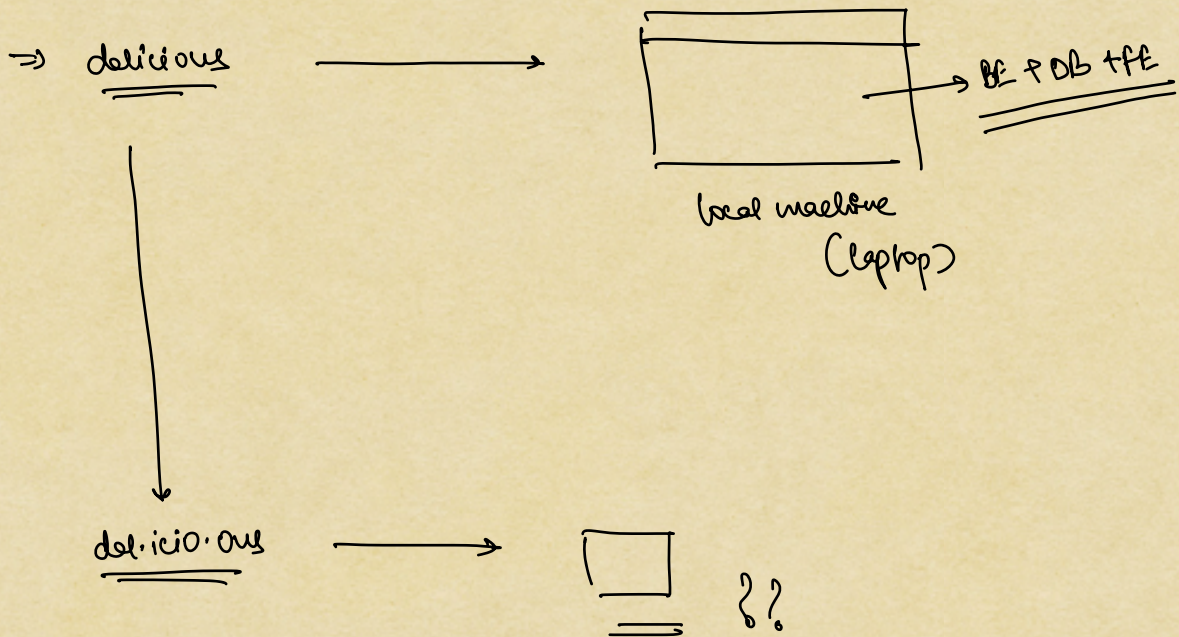
* bookmarking was important

delicious $\rightarrow$ maintained bookmarks which could be retrieved anywhere

|

```

    |
    | → addBookmark( userId, siteURL );
    | → getRecommends( userId );
  
```



⇒ DNS

⇒ bandeep.deu

Q1. Availability

Q2. Assigning to the owner

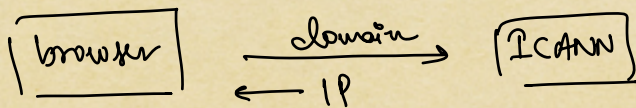
Central Authority

ICANN

maps ⇒ domain Name → list of IPs
 ↓
 owner

(server)

domain Name	IPs	owner

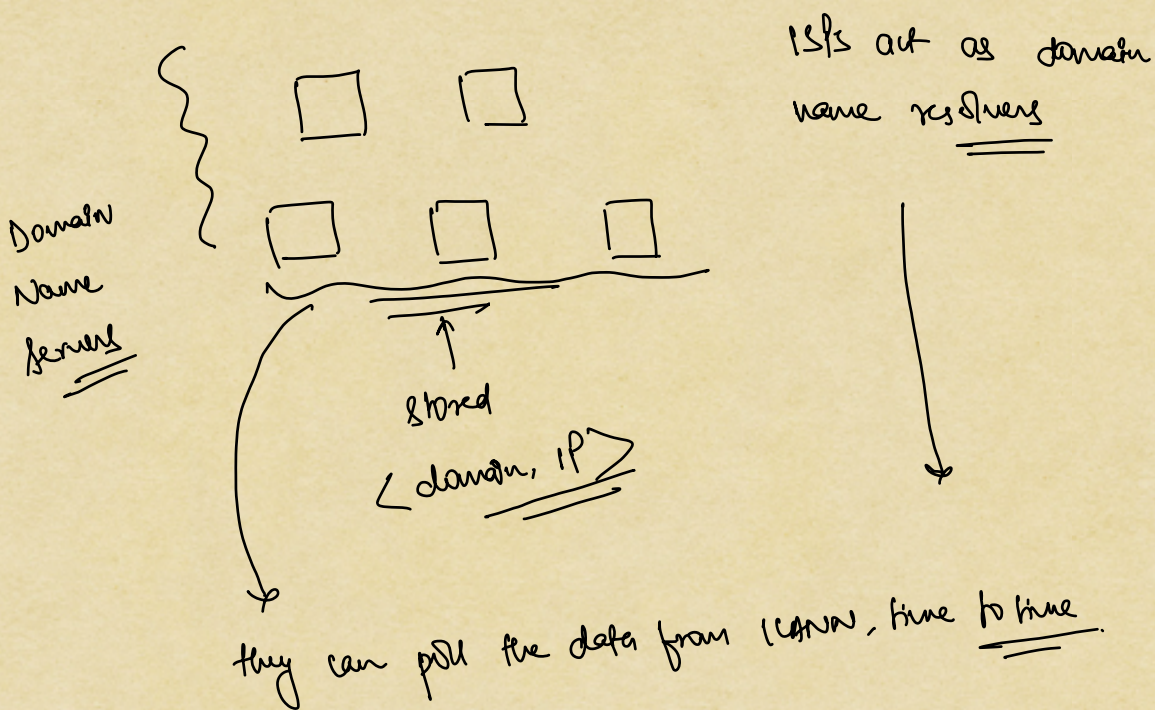


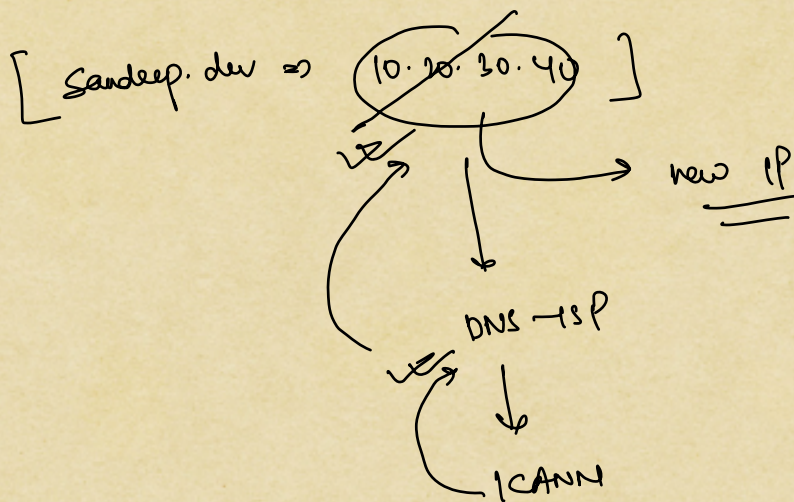
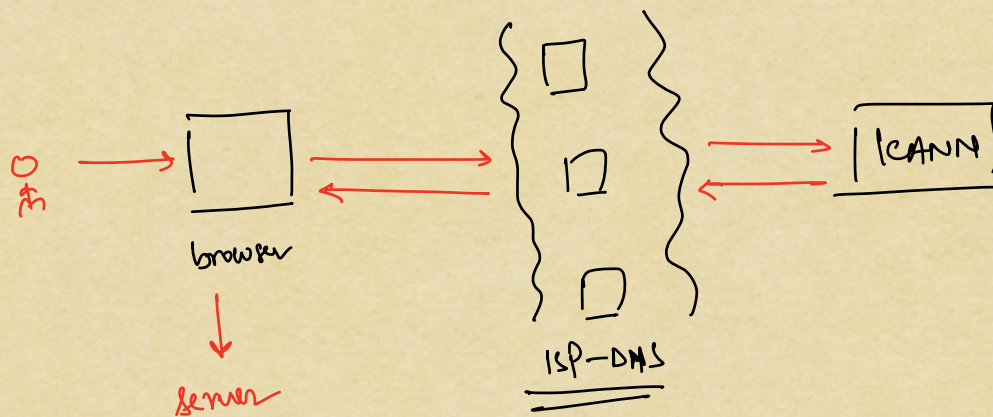
↓
server

* ICANN will not be able to handle
if every case goes through ICANN.

James Webb → 344 spots

* Single Point of Failure (SPoF)



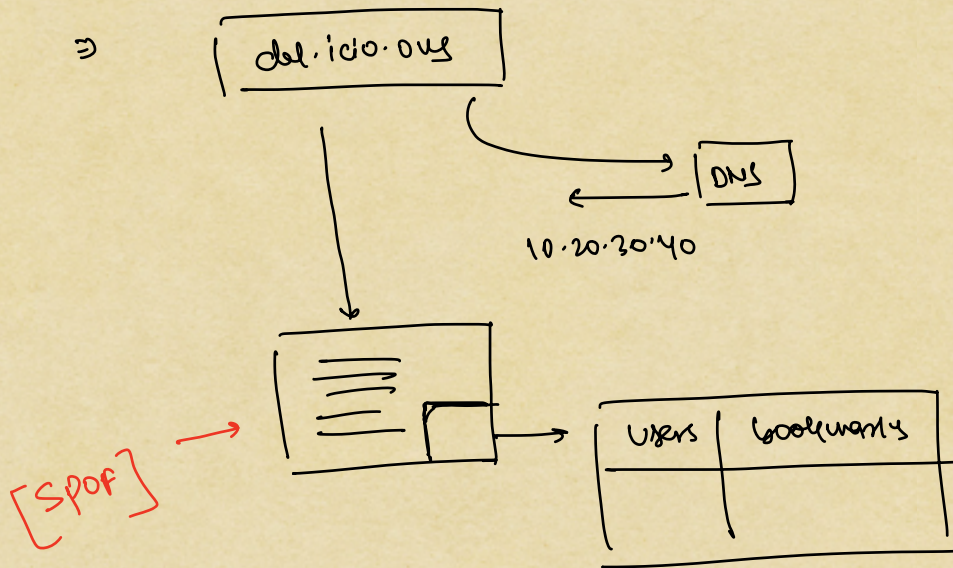
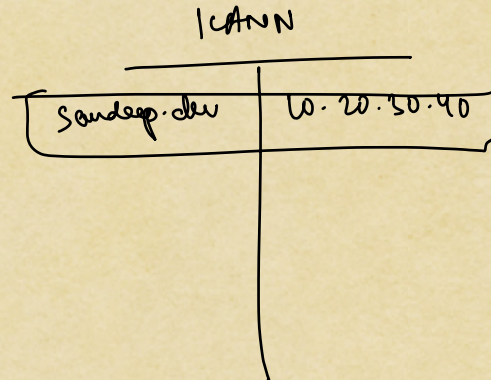


=> dynamic IP vs static IP:

* dynamic => ISP provides us with an IP

↓
gets a new IP
everytime I connect
to internet

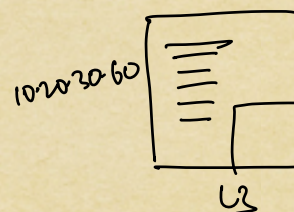
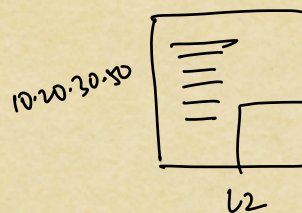
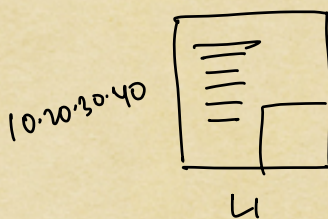
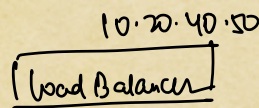
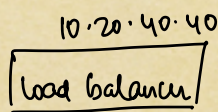
* static => lease connection from ISP. it will give us
a fixed IP addr.



- 1) code deploy → downtime
- 11) laptop reboots → downtime

ICANN

domain Name (list of IP's)



i) Introduced redundancy $\Rightarrow$ NO SPOF

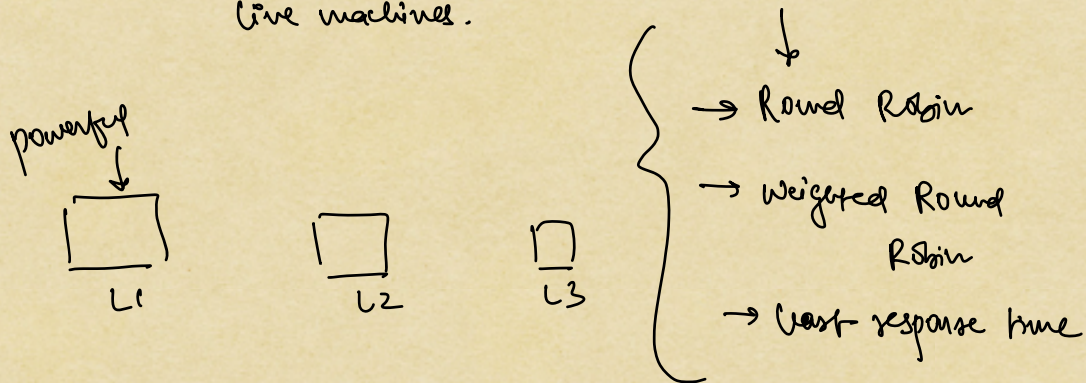
ii) Zero downtime due to code deploy

iii) even if one/two machines go down, system will keep on running

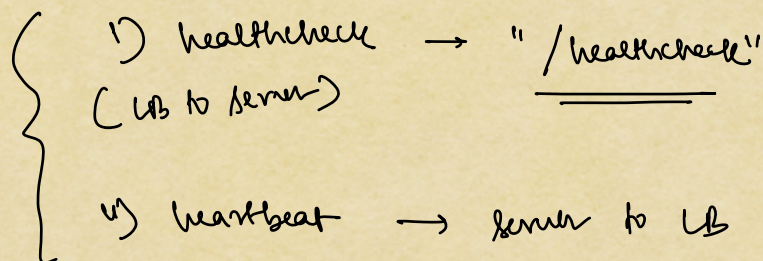
$\Rightarrow$ Responsibilities of LB:

i) route the request among the servers.

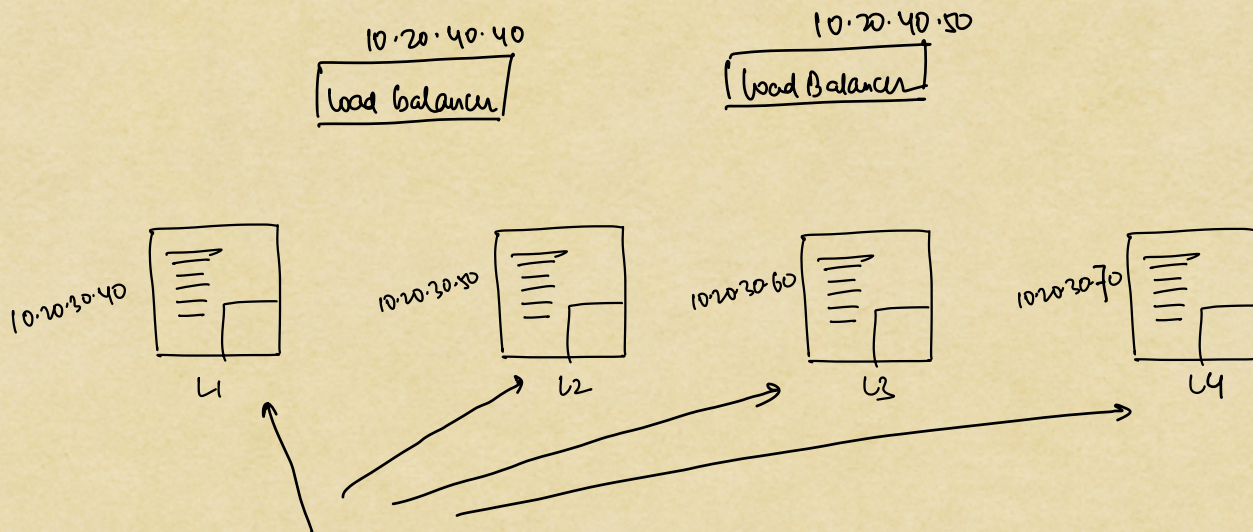
ii) ensure the best effort to send req. to the live machines.



$\Rightarrow$ Server health:



⇒ Consistent Hashing:



if all machines have exact same data, will it be a good or a bad thing?

Q Is data redundancy actually bad?

Ans ⇒ Not always.

Pro ⇒ 1) fail safe

2) saves us from consistency issues

40kB RDD

Cons ⇒ lot of storage is wasted

⇒ lets assume

User Id	usr	timestamp	...

→ 500 bytes

Popular $\Rightarrow$ 1M new bookmarks/day * Reddit King of Death

$\downarrow$
500k x 1M $\Rightarrow$ 500MB/day

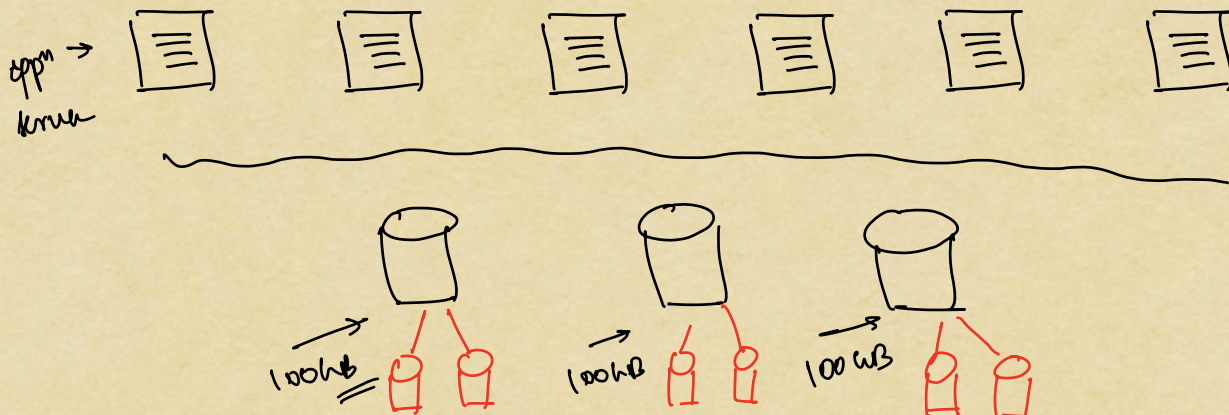
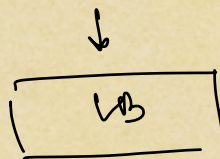
$\downarrow$
40GB HDD $\Rightarrow$ 80 days

* what to do when HDD is full?

* let's divide the tasks

$\rightarrow$ Storage $\Rightarrow$ DBs.
machine
 $\downarrow$
processing $\rightarrow$ appn server

Client



⇒ Same data everywhere ⇒ 100GB

⇒ Split the data ⇒ 100KB ⇒ 300GB

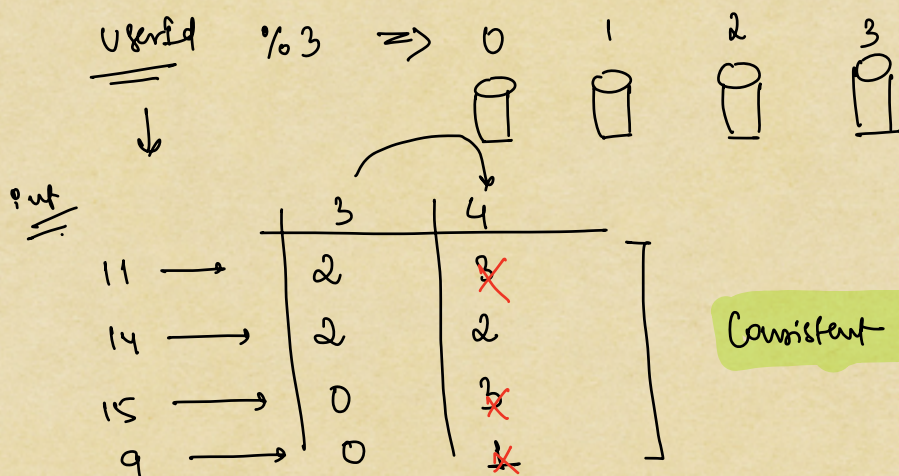
⇒

UserId	Like URL
U1	B1
U2	B2
U1	B3

→ if we randomly split the data, single user bookmarks might also get split, leading to query over all the DBs, hence very slow.



all single user related data should be present in same DB.



Add Bookmark (11, $\rightarrow$)

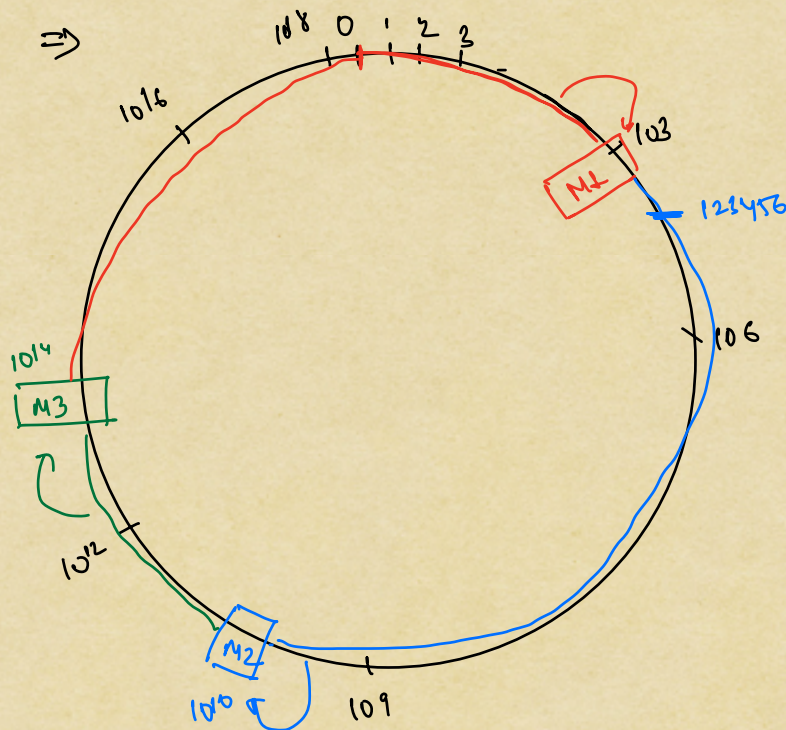
$$\downarrow$$
$$11 \% 3 = \textcircled{2} \rightarrow \text{DB No 2}$$

getBookmark (11)

$$\downarrow$$
$$11 \% 3 = 2 \rightarrow \text{red DB no. 2}$$

$\downarrow$
getBookmark (11)

$$\downarrow$$
$$11 \% 4 = 3 \rightarrow \text{DB } \underline{\underline{\text{no 3}}}$$



hash() $\rightarrow$ [0, 10¹¹]

$$\downarrow$$
$$M1 \Rightarrow 10^3$$

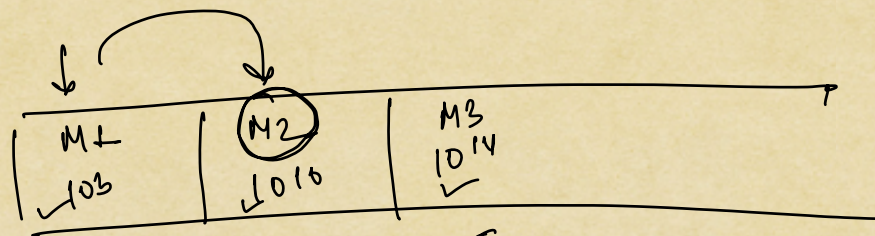
$$\text{hash}(M2) \Rightarrow 10^{10}$$

$$\text{hash}(M3) \Rightarrow 10^{14}$$

$\Rightarrow \text{hash}(\text{userId}) \Rightarrow [0, 10^{18}]$

$\downarrow$
123456 $\Rightarrow$ take the next machine
 $\downarrow$
M2

$\Rightarrow$ how to implement



$\text{hash}(\text{userId}) \Rightarrow$ 123456
 $\downarrow$
10¹⁶

$\text{addBM}(\text{userId}, \text{siteUrl}) \Rightarrow$

$\downarrow$
 $\text{hash}(\text{userId}) \Rightarrow$ number

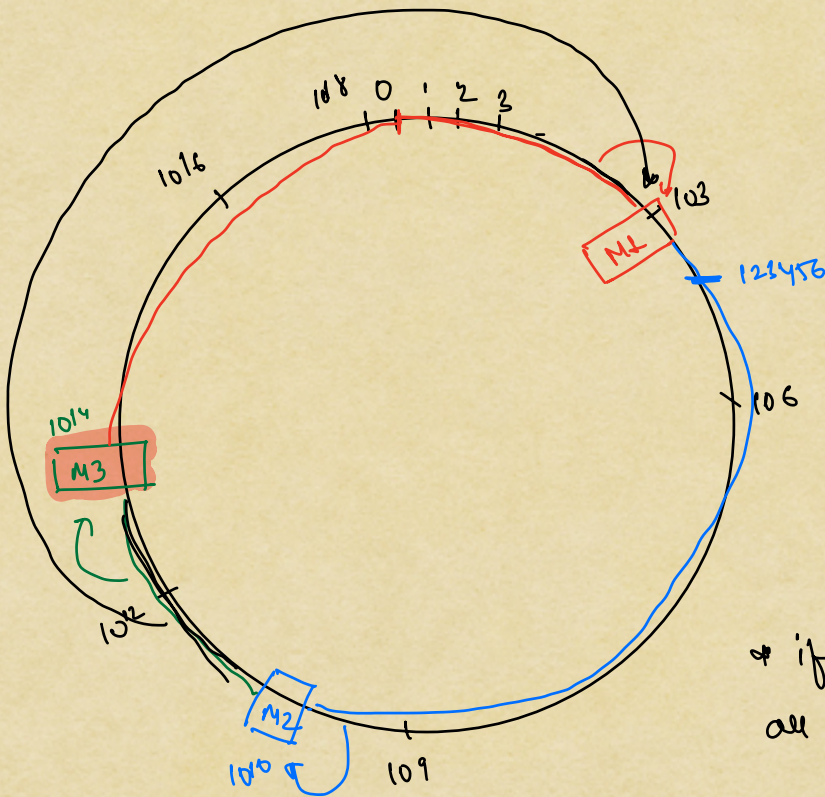
$\text{getBM}(\text{userId})$

$\downarrow$
 $\text{hash}(\text{userId}) \Rightarrow \text{number} \rightarrow$ DB

DB

DB

⇒ Let's assume one machine goes down



* if M3 goes down
all requests will start
routing to M1

* M1 is overloaded ⇒ fail
↓
M2 is overloaded ⇒ fail

Cascading
failure

⇒ lets update algo

lets have more hash functions ⇒ H_1 H_2 H_3

$$H_1(M_1) \rightarrow 10^3$$

$$H_2(M_1) \rightarrow 10^{10}$$

$$H_3(M_1) \rightarrow 10^8$$

$$H_1(M_2) \rightarrow 10^4$$

$$H_2(M_2) \rightarrow 10^{12}$$

$$H_3(M_2) \rightarrow 10^7$$

{
⋮

addBM(vid, vrt)

↓

hash(vid) ⇒ =

getBM(vid)

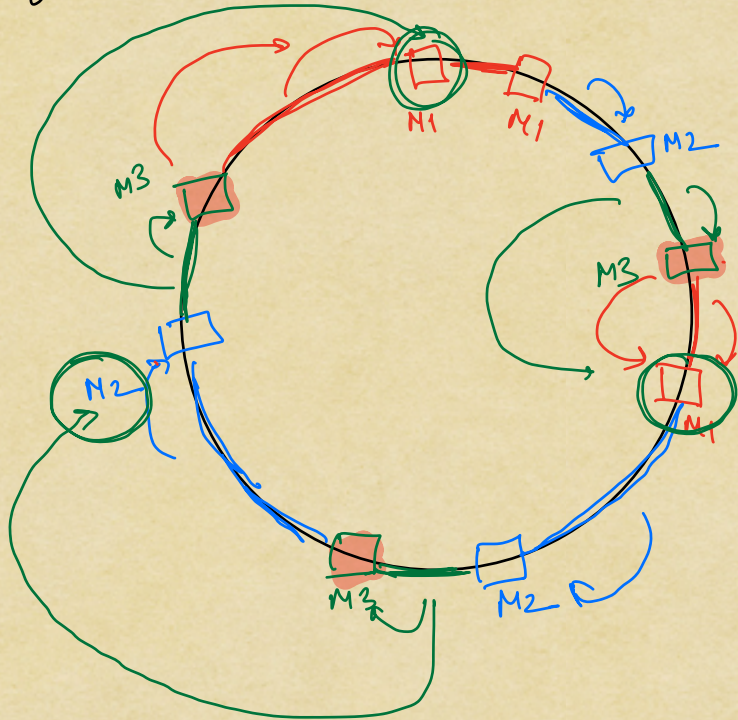
↓

hash(vid) ⇒ =

⇒ assume M_3 goes down(↓)

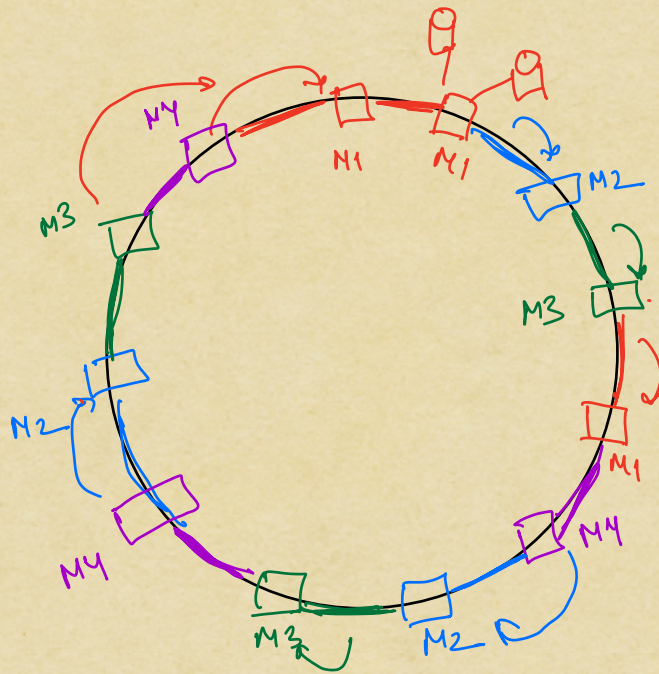
⇒ adding multiple random hash functions

⇒ lead to almost equal distributo
1



↓
lets go to fresh
functions

→ lets add a new machine



→ M4

↓
 $H_1(M_4)$
 $H_2(M_4)$
 $H_3(M_4)$